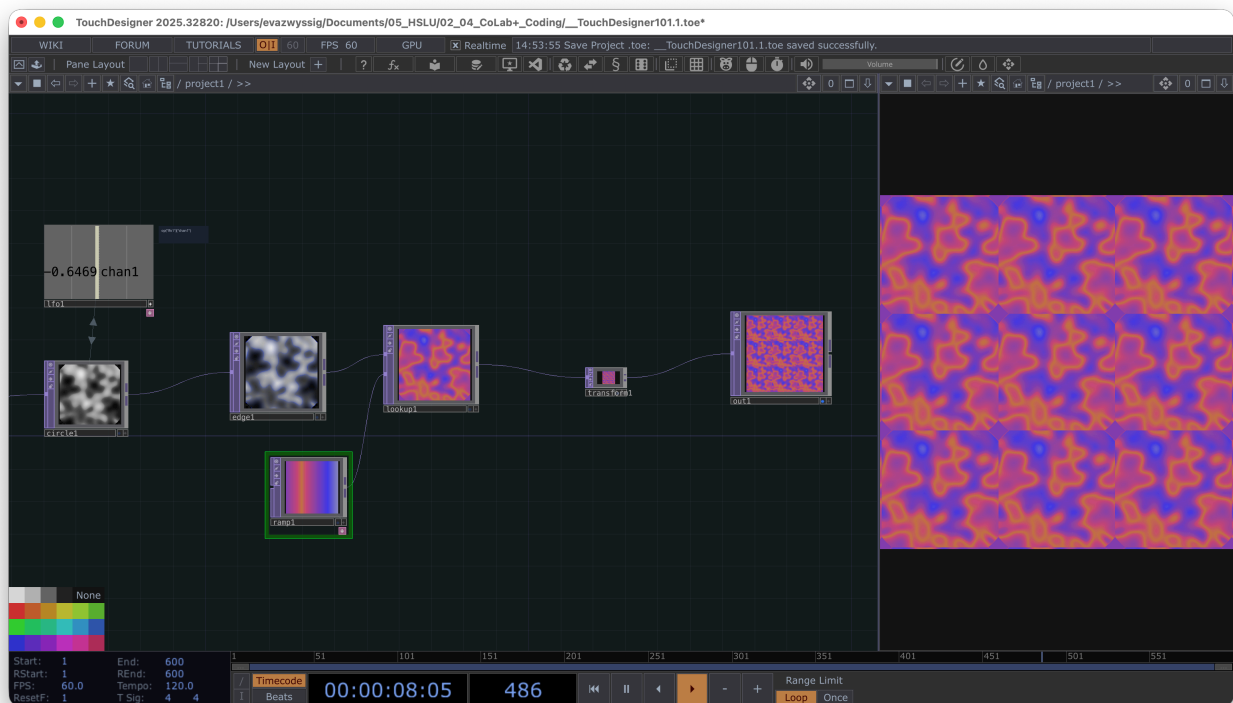
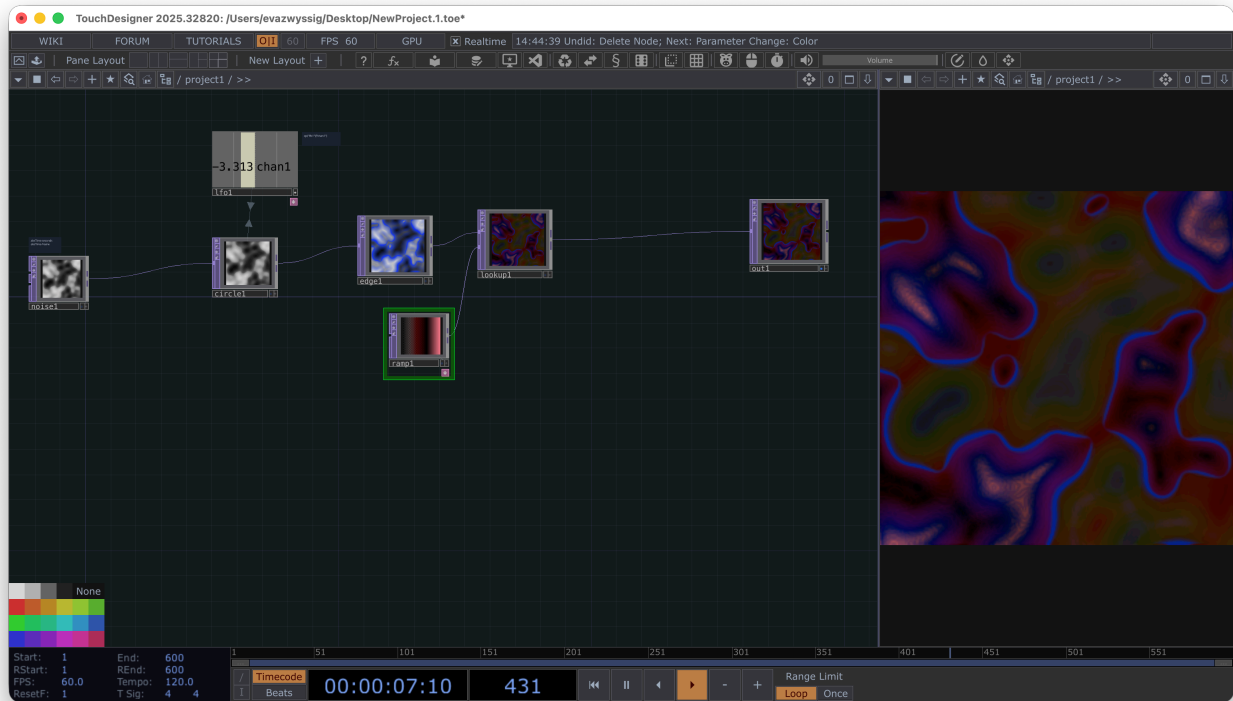
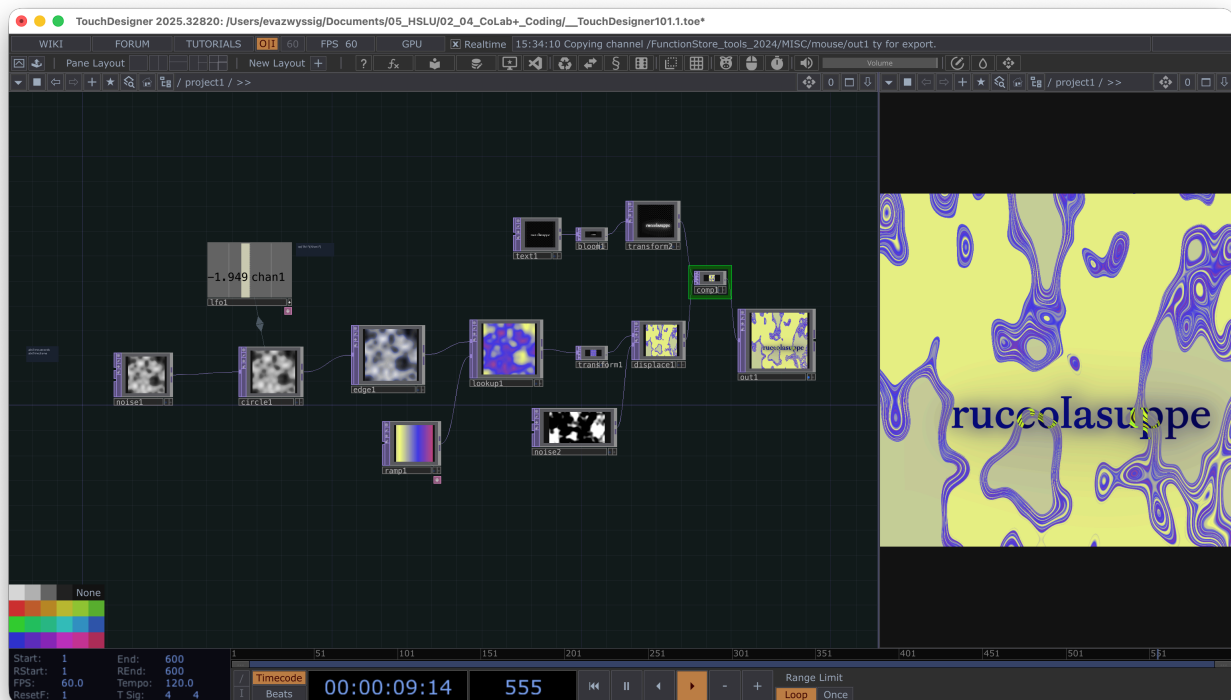
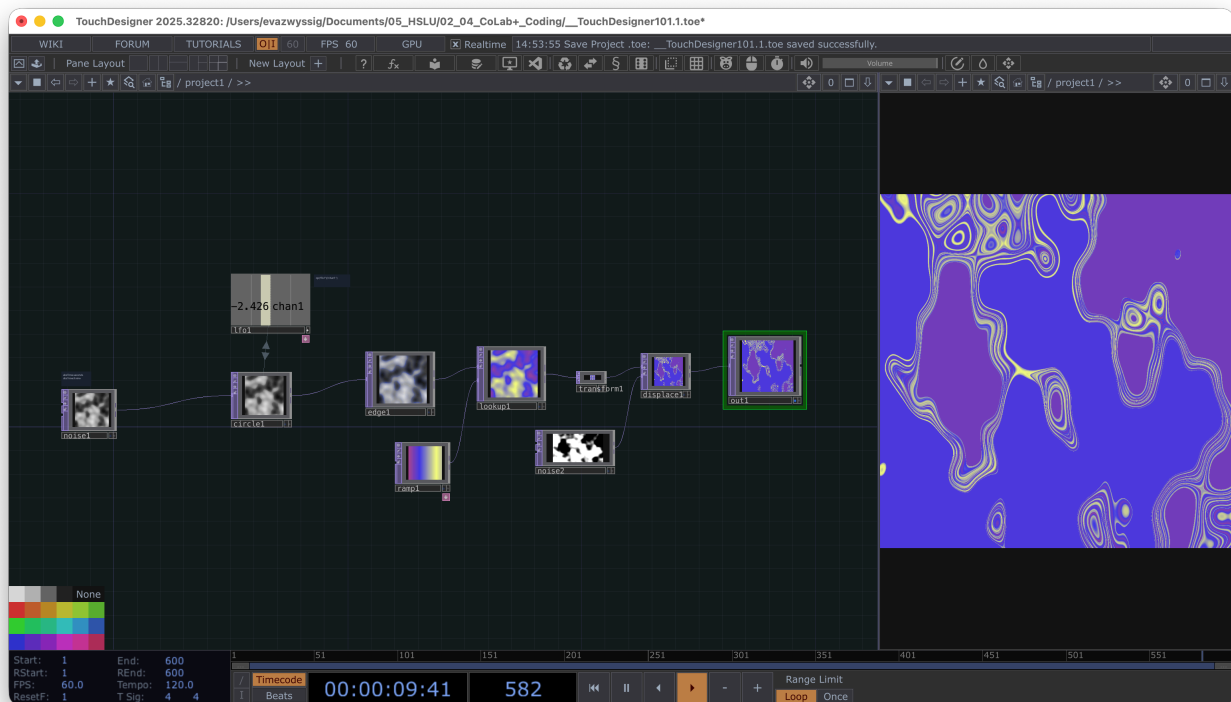
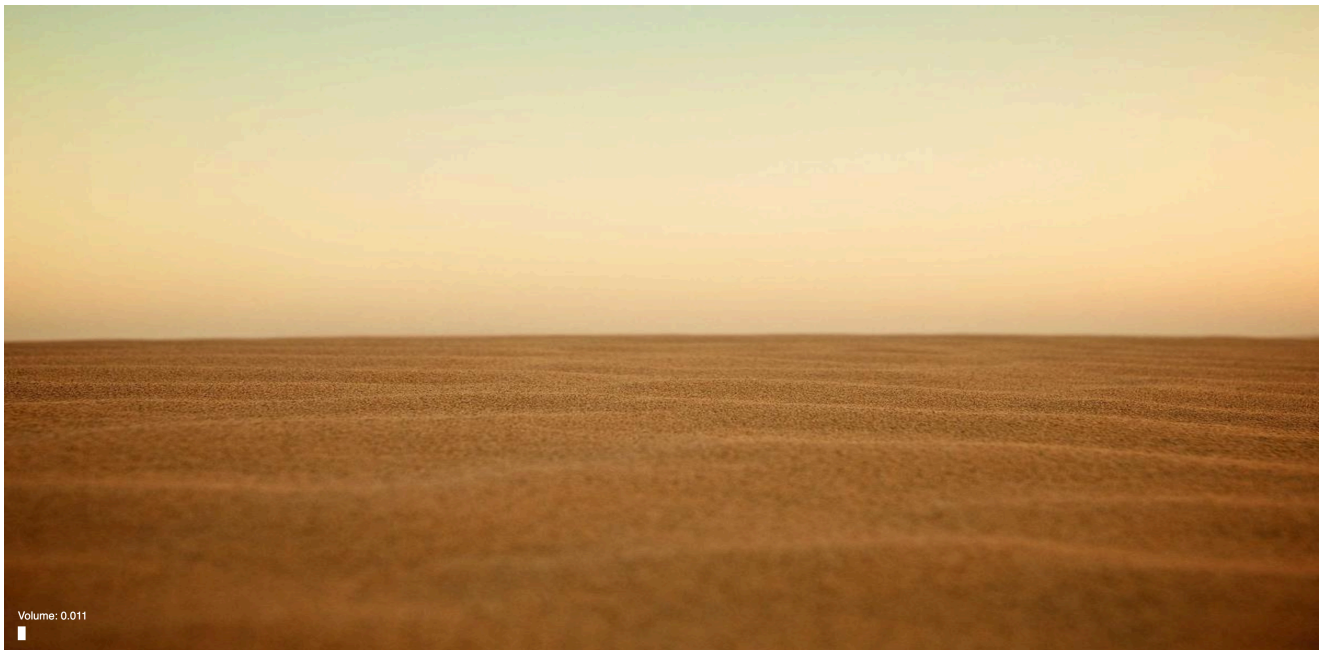


Touchdesigner (Paulinas Hub)





Audioreactive: Random Images (self-study)



```
// {"P5LIVE":{"name":"15_AudioreactiveImageSwap","mod":1779117641778}}
```

```
let mic;  
let currentImg;  
let imgURL = "";  
let threshold = 0.03; // volume trigger  
let cooldown = 1000; // ms between switches  
let lastSwitch = 0;
```

```
function setup() {  
  createCanvas(windowWidth, windowHeight);
```

```
  // start microphone  
  mic = new p5.AudioIn();  
  mic.start();
```

```
  // load first image  
  loadRandomImage();  
}
```

```
function draw() {  
  background(0);  
  
  // draw image fullscreen  
  if (currentImg) {  
    image(currentImg, 0, 0, width, height);  
  }
```

```
  // microphone volume  
  let vol = mic.getLevel();
```

```
  // debug volume meter  
  fill(255);
```

```

noStroke();
rect(20, height - 40, vol * 1000, 20);

fill(255);
textSize(16);
text("Volume: " + nf(vol, 1, 3), 20, height - 50);

// trigger image switch on loud sound
if (vol > threshold && millis() - lastSwitch > cooldown) {
  loadRandomImage();
  lastSwitch = millis();
}
}

function loadRandomImage() {

  // random seed so image changes every time
  let seed = floor(random(100000));

  // random online image source
  imgURL = "https://picsum.photos/seed/" + seed + "/1200/800";

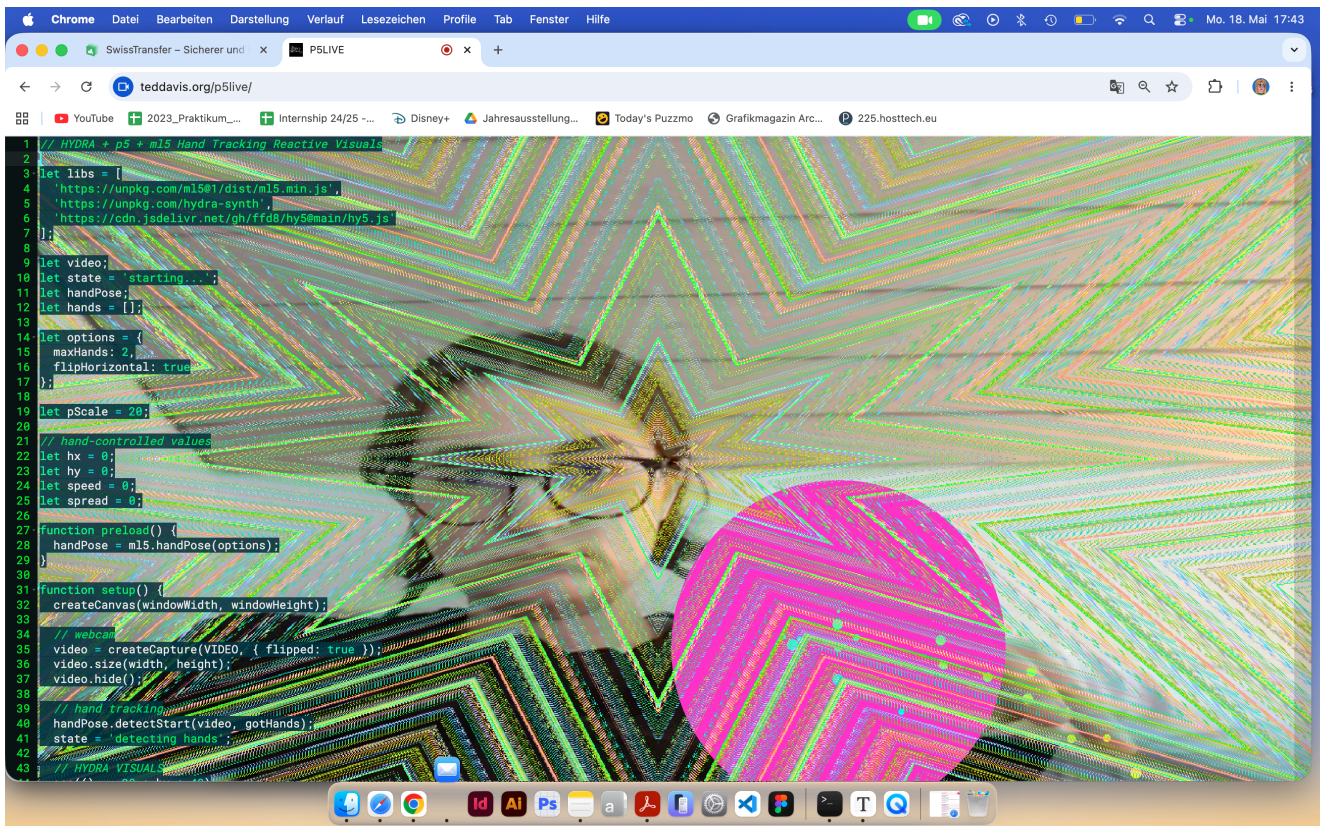
  loadImage(
    imgURL,
    img => {
      currentImg = img;
      console.log("Loaded:", imgURL);
    },
    err => {
      console.error("Image failed:", err);
    }
  );
}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);
}

```

HandPose: Functions linked to Handpositions (self-study)

Honestly I don't fully understand how this works, but it's fun because it also requires a little physical "dance" to use it. Maybe then it should be clearer what each motion does. (up/down, left/right, fingerpinch, turning)



```
let libs = [  
  'https://unpkg.com/ml5@1/dist/ml5.min.js',  
  'https://unpkg.com/hydra-synth',  
  'https://cdn.jsdelivr.net/gh/ffid8/hy5@main/hy5.js'  
];  
  
let video;  
let state = 'starting...';  
let handPose;  
let hands = [];  
  
let options = {  
  maxHands: 2,  
  flipHorizontal: true  
};  
  
let pScale = 20;  
  
// hand-controlled values  
let hx = 0;  
let hy = 0;  
let speed = 0;  
let spread = 0;  
  
function preload() {  
  handPose = ml5.handPose(options);  
}  
  
function setup() {
```

```

createCanvas(windowWidth, windowHeight);

// webcam
video = createCapture(VIDEO, { flipped: true });
video.size(width, height);
video.hide();

// hand tracking
handPose.detectStart(video, gotHands);
state = 'detecting hands';

// HYDRA VISUALS
osc(() => 20 + hx * 40)
  .mult(
    osc(
      () => 1 + speed * 10,
      () => hy * 0.5,
      1
    )
  )
  .modulateRepeat(
    osc(() => 0.1 + spread * 5)
  )
  .colorama(() => hy * 5)
  .rotate(() => speed)
  .kaleid(() => int(3 + spread * 20))
  .luma(() => 0.2 + hy * 0.5)
  .out();
}

function draw() {
  background(0);

  image(video, 0, 0, width, height);

  for (let i = 0; i < hands.length; i++) {

    const hand = hands[i];

    // use index finger tip
    const tip = hand.keypoints[8];

    if (tip) {

      hx = tip.x / width;
      hy = tip.y / height;

      // hand motion speed
      speed = dist(
        tip.x,

```

```

    tip.y,
    pmouseX,
    pmouseY
  ) * 0.01;

  // hand openness
  let thumb = hand.keypoints[4];
  spread = dist(
    thumb.x,
    thumb.y,
    tip.x,
    tip.y
  ) / 300;

  // visual tracker
  fill(255, 0, 200);
  noStroke();
  circle(
    tip.x,
    tip.y,
    50 + speed * 100
  );
}

// draw all points
for (let j = 0; j < hand.keypoints.length; j++) {

  const kp = hand.keypoints[j];

  fill(
    map(j, 0, hand.keypoints.length, 0, 255),
    255,
    map(j, 0, hand.keypoints.length, 255, 0)
  );

  noStroke();

  circle(
    kp.x,
    kp.y,
    pScale * sin(frameCount * 0.1 + j)
  );
}
}

fill(255);
text(state, 20, height - 20);

text("hx: " + nf(hx, 1, 2), 20, 30);
text("hy: " + nf(hy, 1, 2), 20, 50);

```

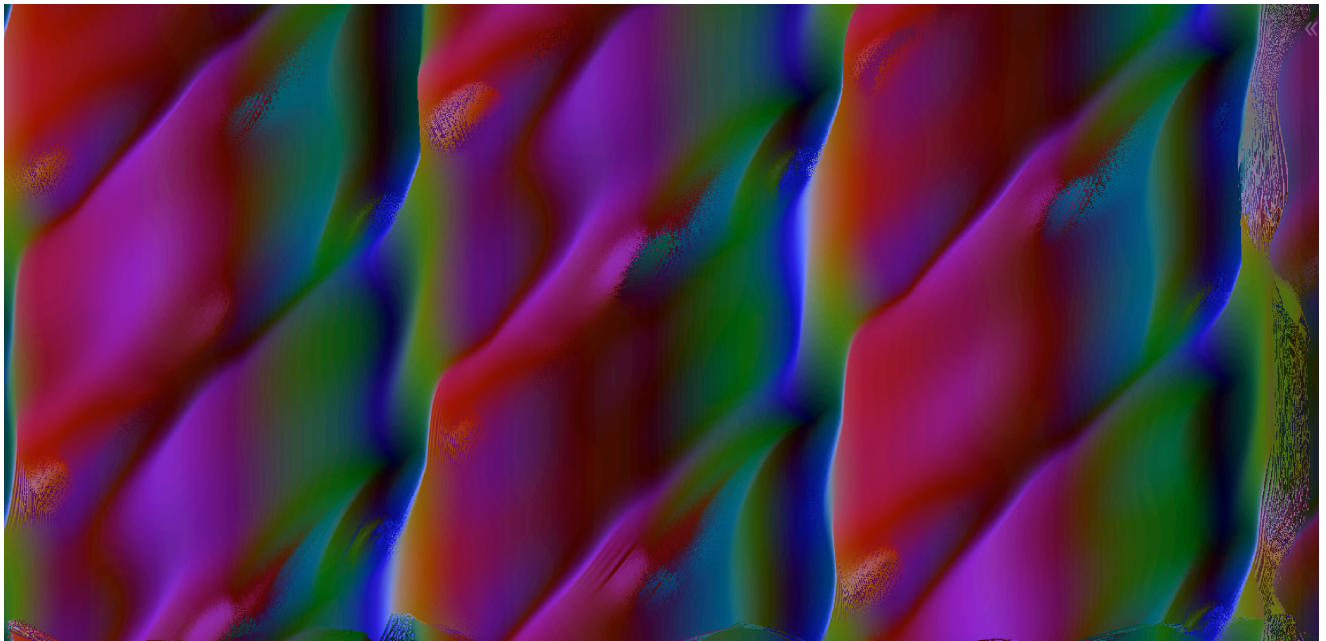
```
text("speed: " + nf(speed,1,2), 20, 70);  
}  
  
function gotHands(results) {  
  hands = results;  
}
```

Audioreactive Hydra (Bass, Mid, High) (self-study)

Bass: Rotation, Oscillator Frequency

Mid: Modulation

High: Color, Distortion



```
let libs = [  
  'https://unpkg.com/hydra-synth',  
  'includes/libs/hydra-synth.js',  
  'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js',  
  'includes/libs/hy5.js'  
]  
  
H.pixelDensity(1)  
  
//audio-reactive values  
let bass = () => a.fft[0] * 0.01  
let mid = () => a.fft[1] * 0.02  
let high = () => a.fft[2] * 0.02  
  
osc(() => 20 + bass() * 0,  
    0.02,  
    () => 0.3 + high() * 0.2)  
  .color(  
    0.3 + high() * 0.2,  
    0.3 + high() * 0.2,  
    0.3 + high() * 0.2
```

```

        () => 0.3 + high(),
        0.1,
        () => 0.8 + mid()
    )
    .rotate(
        () => 0.1 + bass() * 0.4,
        0.1
    )
    .modulate(
        osc(
            () => 30 + mid() * 60)
        .rotate(0.5)
        .add(o0, 0.1),
        () => 0.05 + high() * 0.2
    )
    .add(
        osc(
            () => 20 + high() * 50,
            0.01,
            1
        ).color(0.5, 0.5, 0.2)
    )
    .out(o0)

osc(
    () => 40 + bass() * 90,
    0.05,
    0.7)

    .color(
        0.2, 0.5,
        () => 0.6 + bass()
    )
    .diff(o0)
    .modulate(
        o1,
        () => 0.1 + mid() * 0.5
    )
    .out(o1)

render(o1)

let mic

function setup() {
    createCanvas(windowWidth, windowHeight)
    mic = new p5.AudioIn()
    mic.start()
    userStartAudio()
}

```

```
function draw() {  
  
}
```